



DS 295: Parallel Programming

Project Title:

GPU Acceleration of Recursive Fermi-operator Expansion
within the Framework of a Deep Neural Network

Khadatkar Sameer Raju

Sr. No. 06-18-01-10-51-20-1-17830

*Department of Computational and Data Sciences
Indian Institute of Science, Bangalore, India*

Introduction

- Computation of eigenvalues of a matrix forms the basis of solving many scientific problems.
- One of its applications is in Quantum Mechanical calculations using Density Functional Theory.
- In this we need to solve this following non-linear eigenvalue problem.

$$\left(-\frac{1}{2} \nabla^2 + V_{\text{eff}}(\rho) \right) \psi_i = \epsilon_i \psi_i, \quad \text{--- (I)}$$

- The above calculation requires the computation of single particle electron density (ρ).
- Numerically ρ is computed by solving the discretised eigenvalue problem corresponding to equation (I)

Fermi-operator Expansion

An alternate approach which avoids explicit diagonalization of discretised system matrix(H) is to use density matrix Fermi-operator expansion.

$$D = (e^{\beta(H-\mu I)} + I)^{-1}$$

Recursive Fermi-operator expansion

$$D = \theta(\mu I - H) \approx f_m(f_{m-1}(\dots f_0(H) \dots)).$$

Second Order spectral projection Polynomials

$$X_{i+1} = f_i(X_i) = X_i \pm (X_i - X_i^2)$$

Neural Network Formulation

N_{occ} , Number of occupied states or orbitals
 ϵ , small number close (or equal) to 0
 H , Orthonormalized Hamiltonian
 ϵ_1, ϵ_N , Spectral bound estimates of H
 W_n , Observable operator matrix, e.g. $W_n = H$
 $X_0 = H$, Input layer
 $W_0 = -(\epsilon_N - \epsilon_1)^{-1}I$, $B_0 = \epsilon_N(\epsilon_N - \epsilon_1)^{-1}I$
 $S_0 = W_0X_0 + B_0$, $N_S = \text{Tr}[S_0]$, Occupation of S_0
 $n = 0$, Number of layers

while Not Converged **do**

$n = n + 1$

$X_n = f(S_{n-1})$,

$N_X = \text{Tr}[X_n]$

$\text{IdErr}_n = N_S - N_X$, Idempotency error estimate

$\sigma_n = \text{Sign}(|2N_S - N_X - N_{occ}| - |N_X - N_{occ}| - \sigma_{n-1}\epsilon)$

$W_n = \sigma_n I$, $B_n = (I - W_n)S_{n-1}$

$S_n = W_nX_n + B_n$

$N_S = \frac{N_X}{\sigma_n} + (1 - \sigma_n)N_S$, Updated occupation of S_n

if $\text{IdErr}_n \leq 0$ **then**

 Converged = **true**

else if $n > 2$ **and** $\sigma_{n-1} \neq \sigma_{n-2}$ **and** $\text{IdErr}_n > 4.5 \times (\text{IdErr}_{n-2})^2$ **then**

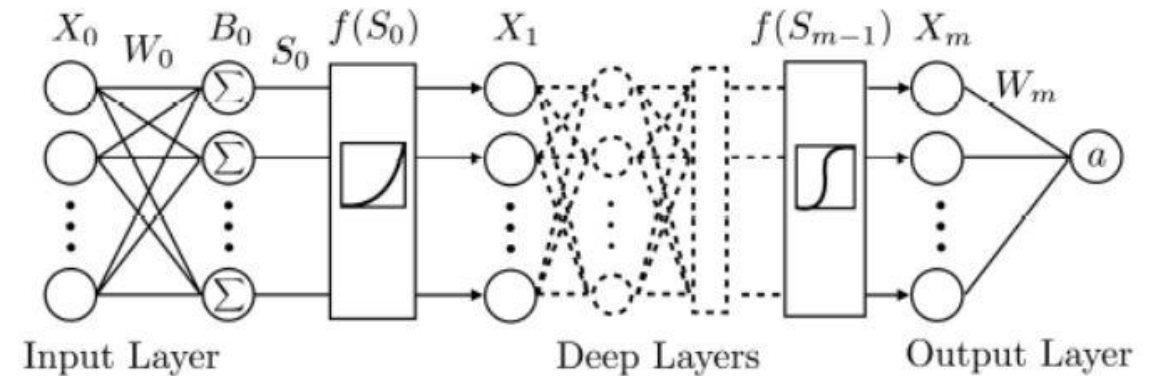
 Converged = **true**

end if

end while

$D = f_{\text{final}}(S_{m-1})$, final X_m layer

$\alpha = \text{Tr}[DW_m]$, Output observable



This algorithm can be naturally mapped to the above neural network structure

$$X_i = f(\dots f(W_i f(W_0 X_0 + B_0) + B_1) \dots)$$

$$f(S_0) = S_0^2$$

Motivation

- Often this system discretized matrix can be large and computing this one particle density matrix using the recursive Fermi-operator expansion techniques can become time-consuming.
- We can accelerate this computation by making use of parallel architectures and reduce the execution time of this calculations.
- The paper which I am referring has an implementation of this method on GPUs using python.
- But, they didn't explicitly mentioned how to implement this method on GPUs.

Methodology

N_{occ} , Number of occupied states or orbitals

ϵ , small number close (or equal) to 0

H , Orthonormalized Hamiltonian

ϵ_1, ϵ_N , Spectral bound estimates of H

W_n , Observable operator matrix, e.g. $W_n = H$

$X_0 = H$, Input layer

$W_0 = -(\epsilon_N - \epsilon_1)^{-1}I$, $B_0 = \epsilon_N(\epsilon_N - \epsilon_1)^{-1}I$

$S_0 = W_0X_0 + B_0$, $N_S = \text{Tr}[S_0]$, Occupation of S_0

$n = 0$, Number of layers

while Not Converged **do**

$n = n + 1$

$X_n = f(S_{n-1})$,

$N_X = \text{Tr}[X_n]$

$\text{IdErr}_n = N_S - N_X$, Idempotency error estimate

$\sigma_n = \text{Sign}(|2N_S - N_X - N_{occ}| - |N_X - N_{occ}| - \sigma_{n-1}\epsilon)$

$W_n = \sigma_n I$, $B_n = (I - W_n)S_{n-1}$

$S_n = W_nX_n + B_n$

$N_S = W_nN_X + (1 - \sigma_n)N_S$, Updated occupation of S_n

if $\text{IdE } \sigma_n \leq 0$ **then**

Converged = **true**

else if $n > 2$ **and** $\sigma_{n-1} \neq \sigma_{n-2}$ **and** $\text{IdErr}_n > 4.5 \times (\text{IdErr}_{n-2})^2$ **then**

Converged = **true**

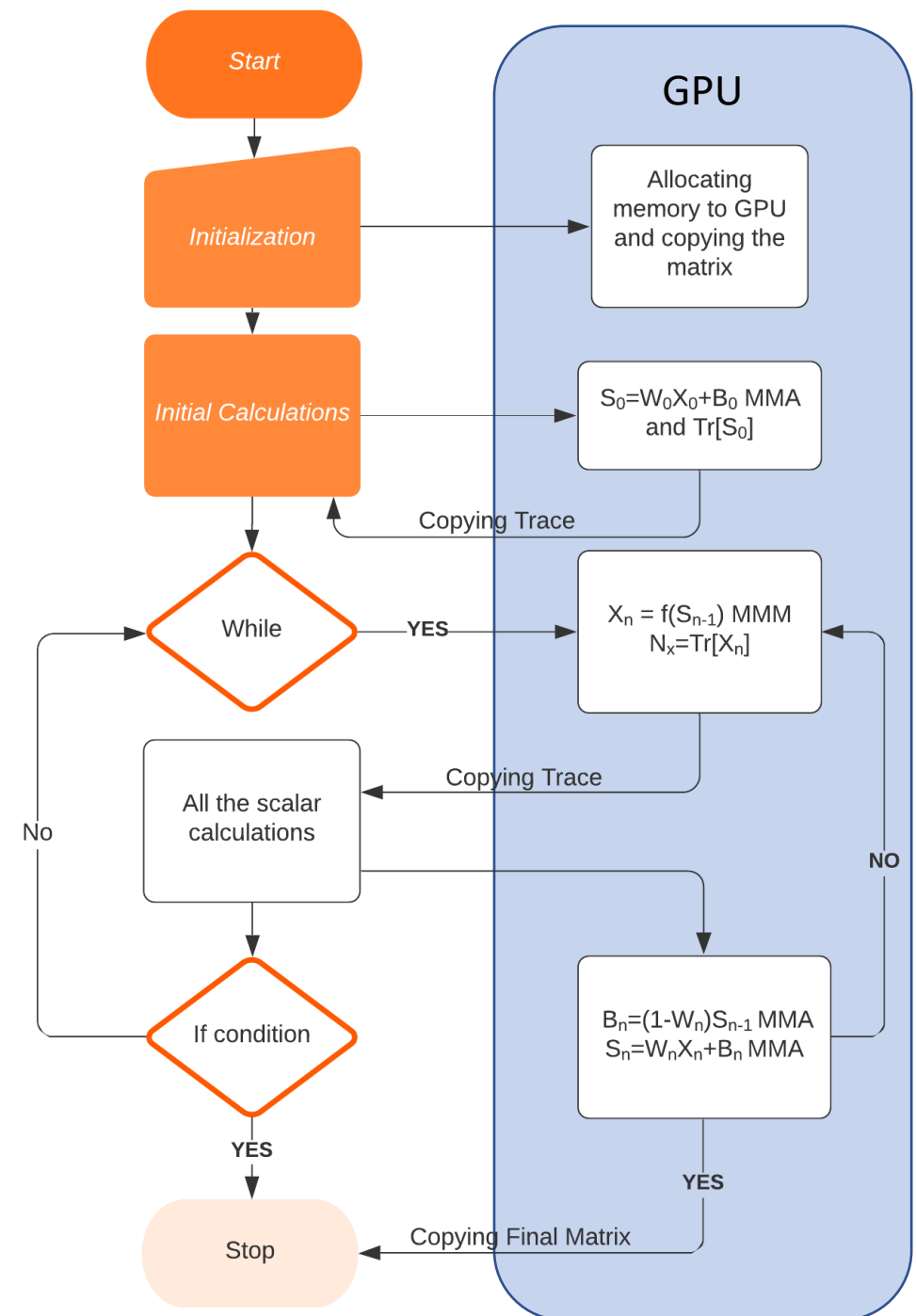
end if

end while

$D = f_{\text{final}}(S_{m-1})$, final X_m layer

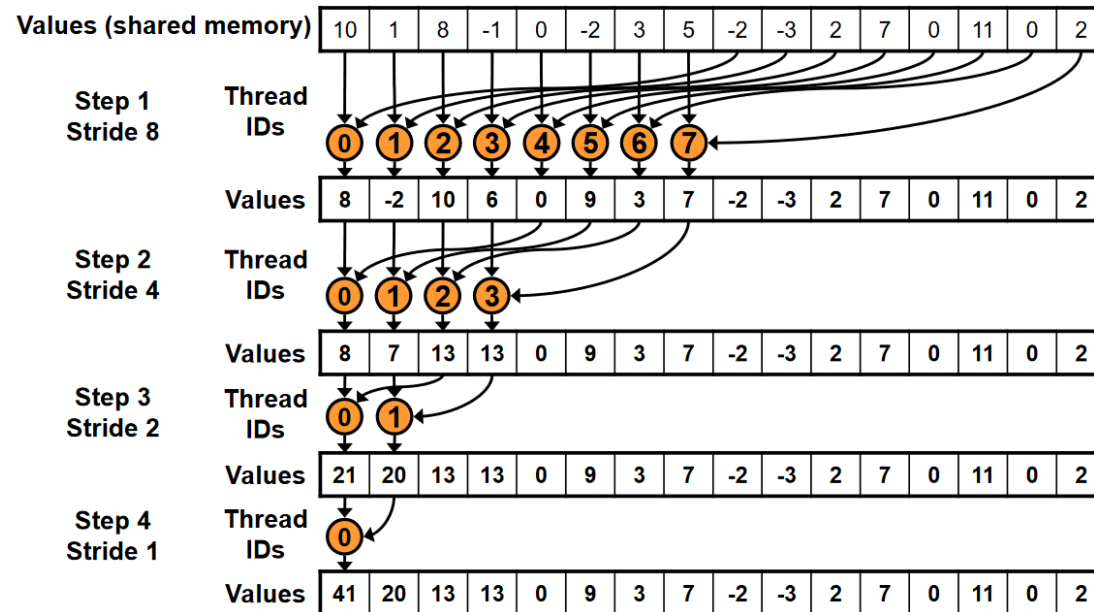
$\alpha = \text{Tr}[DW_m]$, Output observable

18-06-2021



Calculation of trace

- It required 2 kernels 1st for calculating the addition of diagonal elements using the reduction operation of size equal to block size and save it in new array.
- 2nd for calculating the sum of elements of this new array again by reduction (If the size of matrix was greater than our block size).



Reduction operation

Calculation of MM Addition and Multiplication

This calculations was done using cuBLAS library of NVIDIA.

Function used were:

```
cublasSgeam(cublasHandle_t handle,  
            cublasOperation_t transa, cublasOperation_t transb, int m, int n,  
            const float *alpha, const float *A, int lda,  
            const float *beta, const float *B, int ldb, float *C, int ldc)  
cublasSgemm(cublasHandle_t handle,  
            cublasOperation_t transa, cublasOperation_t transb, int m, int n,  
            int k, const float *alpha, const float *A, int lda,  
            const float *B, int ldb, const float *beta, float *C, int ldc)
```


Mixed Precision Algorithm followed by tensor cores

X Input matrix in single precision

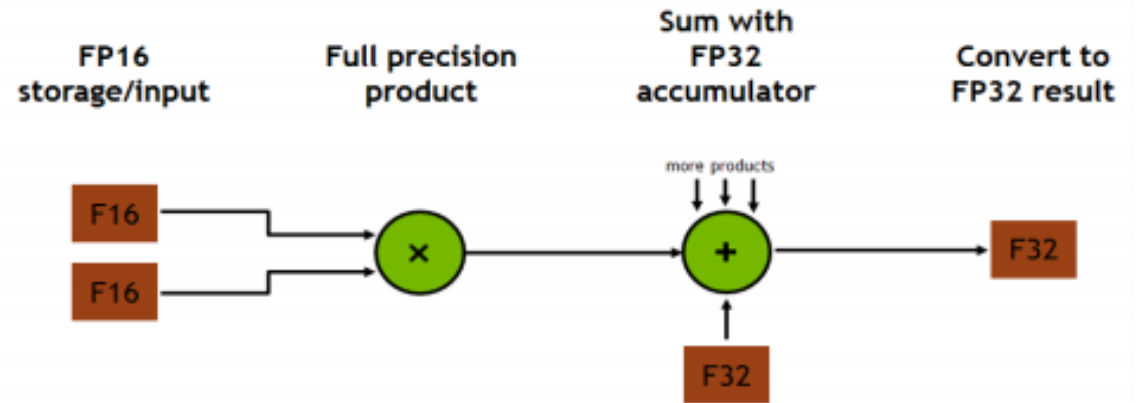
$$X^{(0)} = \text{FP16}[X]$$

$$X^{(1)} = \text{FP16}[X - X^{(0)}]$$

$$A = \text{FP32}[X^{(0)} \times X^{(0)}] \quad \text{Tensor core multiplication}$$

$$B = \text{FP32}[X^{(0)} \times X^{(1)}] \quad \text{Tensor core multiplication}$$

$$f(X) = X^2 \approx \text{FP32}[A + B + B^T]$$



Ref. Nvidia

To use tensor cores we set the math mode of handle of cublas to `CUBLAS_TENSOR_OP_MATH`

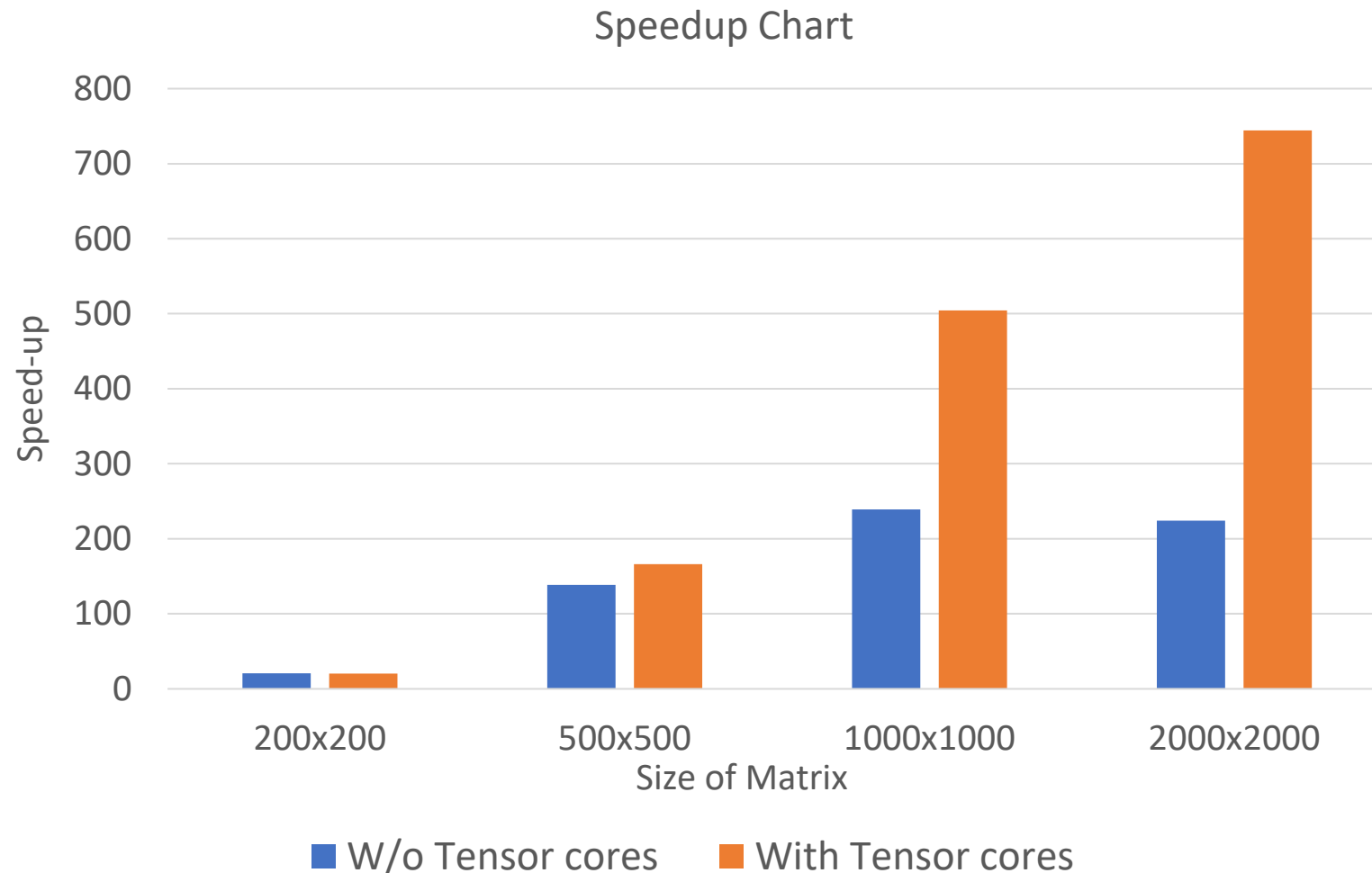
Using the function

```
cublasSetMathMode(handle, CUBLAS_TENSOR_OP_MATH);
```

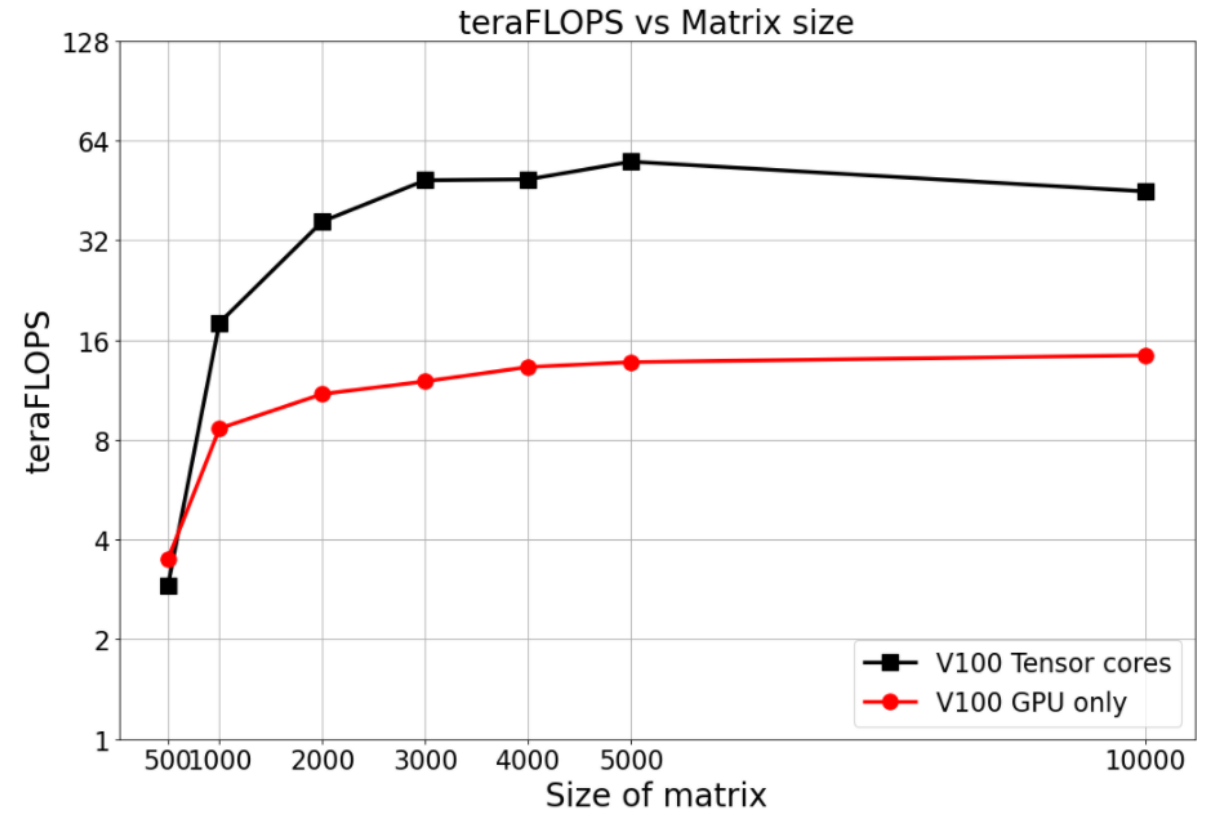
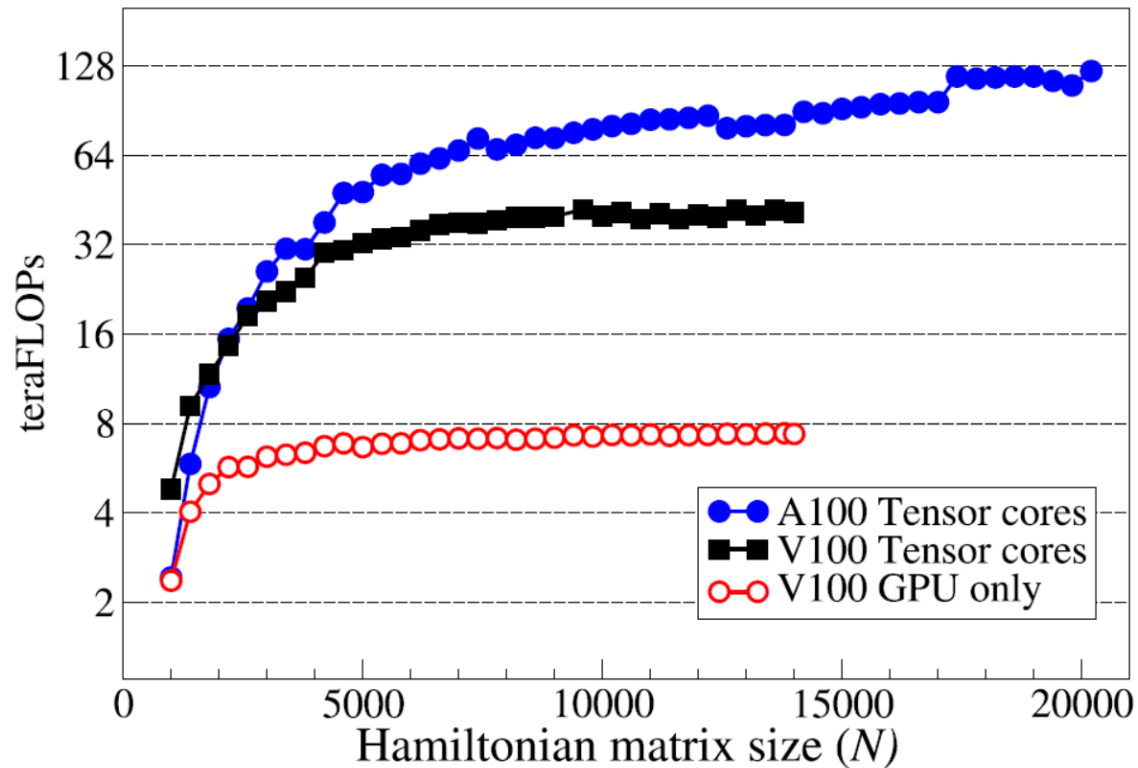
For normal operations `CUBLAS_PEDANTIC_MATH` mode is used

Experiments and Results

Speedups w.r.t Sequential CPU python Code provided in the supporting material of the paper I am referring to



Comparison with existing Implementation



How I calculated teraFlops

1. Got number of flop using nvprof --metrics flop_count_sp command.

```
==145133== Profiling application: ./1stopt 1000 1 0
==145133== Profiling result:
==145133== Metric result:
Invocations      Metric Name      Metric Description      Min      Max      Avg
Device "Tesla V100-SXM2-32GB (0)"
  Kernel: void scal_kernel<float, float, int=1, bool=0, int=6, int=5, int=5, int=3>(cublasTransposeParams<float>, float const *, float*, float const *)
    13      flop_count_sp  Floating Point Operations(Single Precision)      1024000      1024000      1024000
  Kernel: trace(float*, int, float*)
    36      flop_count_sp  Floating Point Operations(Single Precision)           1023           1023           1023
  Kernel: void scal_kernel<float, float, int=1, bool=1, int=6, int=5, int=5, int=3>(cublasTransposeParams<float>, float const *, float*, float const *)
    21      flop_count_sp  Floating Point Operations(Single Precision)              0              0              0
  Kernel: void geam_kernel<float, bool=0, bool=0, bool=0, bool=0, int=6, int=5, int=3, bool=1>(cublasGeamParams<float>, float const *, float const *, float*, float const *, float const *)
    35      flop_count_sp  Floating Point Operations(Single Precision)      3072000      3072000      3072000
  Kernel: volta_sgemv_32x128_nn
    35      flop_count_sp  Floating Point Operations(Single Precision) 2121269248 2121269248 2121269248
```

2. Then dividing total flop with execution time of algorithm to calculate flop/second

Conclusion

- My C++ implementation was faster in most cases than the existing implementation in the paper when compared on V100 GPUs.
- The main bottleneck in the algorithm was the activation function i.e. the squaring of matrix.
- Mixed precision approach gave good speedup compared to the normal MM multiplication.

Thank You

Reference:

Mixed Precision Fermi-Operator Expansion on Tensor Cores from a Machine Learning Perspective.

Joshua Finkelstein, Justin S. Smith, Susan M. Mniszewski, Kipton Barros, Christian F. A. Negre, Emanuel H. Rubensson, and Anders M. N. Niklasson. *Journal of Chemical Theory and Computation* 2021 17 (4), 2256-2265.

DOI: 10.1021/acs.jctc.1c00057

<https://doi.org/10.1021/acs.jctc.1c00057>